decision tree

mahmood

May 27, 2023

note
this document and others can be found on my website
`https://mahmoodsheikh36.github.io/`

[?]

Definition 0.0.1. a **decision tree** represents a function that takes as input a vector of attribute values and returns a "decision"–a single output value. the input and output values can be discrete or continuous

Characteristic 1. for a wide variety of problems, the decision tree yields a nice, concise result. but some functions cannot be represented concisely, for example, the majority function, which returns true if and only if more than half of the inputs are true, requires an exponentially large decision tree.

1 inducing boolean decision trees from examples code:math

no matter how we measure size, it is an intractable problem to find the smallest consistent tree; there is no way to efficiently search through the 2^{2^n} trees. with some simple heuristics, however, we can find a good approximate solution: a small (but not smallest) consistent tree. the DECISION-TREE-LEARNING algorithm adopts a greedy divide-and-conquer strategy: always test the most important attribute first. this test divides the problem up into smaller subproblems that can then be solved recursively. by most important attribute

we mean the one that makes the most difference to the classification of an example. that way, we hope to get to the correct classification with a small number of tests, meaning that all paths in the tree will be short and the tree as a whole will be shallow.
in general, after the first attribute test splits up the examples, each outcome is a new decision tree learning problem in itself, with fewer examples and one less attribute. there are four cases to consider for these recursive problems:

1. if the remaining examples are all positive (or all negative), then we are done: we can answer Yes or No

2. if there are some positive and some negative examples, then choose the best attribute to split them.

3. if there are no examples left, it means that no example has been observed for this combination of attribute values, and we return a default value calculated from the plurality classification of all the examples that were used in constructing the nodes parent. these are passed along in the variable $parent_{examples}$

4. if there are no attributes left, but both positive and negative examples, it means that these examples have exactly the same description, but different classifications. this can happen because there is an error or noise in the data; because the domain is nondeterministic; or because we cant observe an attribute that would distinguish the examples. the best we can do is return the plurality classification of the remaining examples.

```
Output: decision tree
DECISION-TREE-LEARNING(examples, attributes, parent_examples):
    if examples is empty then
        return PLURALITY-VALUE(parent_examples)
    else if all examples have the same classification then
        return the classification
    else if attributes is empty then
        return PLURALITY-VALUE(examples)
    else
        A ← argmax_{a∈attributes} IMPORTANCE(a, examples)
        tree ← a new decision tree with root test A
        for v_k ∈ A do
            exs ← e : e ∈ examples ∧ e.A = v_k
            subtree ← DECISION-TREE-LEARNING(exs, attributes-A,
            examples)
            add a branch to tree with label (A = v_k) and subtree subtree
        return tree
```

an attribute a with d distinct values divides the training set E into subsets $E_1, \ldots, E_d$. each subset E_k has p_k positive examples and n_k negative examples, so if we go along that branch, we will need an additional $B(p_k/(p_k+n_k))$ bits of information to answer the question. a randomly chosen example from the training set has the k'th value for the attribute with probability $(p_k + n_k)/(p + n)$, so the expected entropy remaining after testing attribute A is

$$Remainder(A) = \sum_{k=1}^{d} \frac{p_k + n_k}{p + n} B\left(\frac{p_k}{p_k + n_k}\right)$$

note

this confused me at first, why multiply probability by entropy? and why would the result be entropy?
turns out, the probability is giving "weight" to the entropy, as the entropy of a more probable branch is more important than a less probable one

the information gain from the attribute test on A is the expected reduction in entropy:

$$Gain(A) = B\left(\frac{p}{p + n}\right) - Remainder(A)$$

which is exactly what we need to implement the IMPORTANCE function, as the attribute that gives the most information gain is the one that should be split on

```
(defun decision-tree-learning (examples attributes
   parent-examples)
  "generate a boolean decision tree given a set of examples"
  ;; if no examples left, return the most frequent
     classification of the parent node's examples
  (if (not examples)
      (return-from decision-tree-learning (list 'answer (
         plurality-classification parent-examples))))
  ;; if all examples have the same classification, return it
  (let ((first-example (car examples))
        (same-classification t))
    (loop for example in examples
          do (if (not (equal (getf example 'goal) (getf
             first-example 'goal)))
                (setf same-classification nil)))
    (if same-classification
        (return-from decision-tree-learning (list 'answer (getf
           first-example 'goal)))))
  ;; if no attributes left, we're done splitting, just return
     the most frequent classification
  (if (not attributes)
      (return-from decision-tree-learning (list 'answer (
         plurality-classification examples))))
  ;; find the attribute to split on, the one that gives the
     most information gain
  (let ((most-important-attribute (car attributes)))
    (loop for attribute in attributes
          do (let ((current-entropy (
             boolean-random-variable-entropy (/ (length (
             positive-examples examples)) (length examples)))))
               (if (> (attribute-information-gain attribute
                  examples current-entropy) (
                  attribute-information-gain
                  most-important-attribute examples
                  current-entropy))
```

```
                  (setf most-important-attribute attribute))))
    ;; create the node and its children
    (let ((tree (list 'attribute-name most-important-attribute)
       )
         (this-attributes-possible-values (
            possible-attribute-values most-important-attribute
             examples)))
      (loop for attribute-value in
         this-attributes-possible-values
           do (let ((examples-with-this-attribute-value '()))
                (loop for example in examples
                      do (if (equal (getf example
                         most-important-attribute)
                         attribute-value)
                            (push example
                               examples-with-this-attribute-value
                               )))
                (let ((subtree (decision-tree-learning
                   examples-with-this-attribute-value (remove
                   most-important-attribute attributes)
                   examples)))
                  (setf (getf subtree 'branch-value) (format nil
                      "~a=~a" most-important-attribute
                     attribute-value))
                  (push subtree (getf tree 'children)))))
      tree)))

(defun plurality-classification (examples)
  "get the most frequent answer, yes/no, in a set of examples"
  (let ((yes-counter 0)
        (no-counter 0))
    (loop for example in examples
         do (if (equal (getf example 'goal) "yes")
                (incf yes-counter)
                (incf no-counter)))
    (if (> yes-counter no-counter)
        "yes"
        "no")))

(defun possible-attribute-values (attribute examples)
  "get a list of the values that an attribute can hold"
```

```
  (let ((possible-values nil))
    (loop for example in examples
          do (pushnew (getf example attribute) possible-values :
             test 'equal)) ;; use equal to compare
    possible-values))

(defun boolean-random-variable-entropy (probability)
  (cond ((equal probability 1) 0) ;; always true
        ((equal probability 0) 0) ;; always false
        (t (- (+ (* probability (log probability 2)) (* (- 1
           probability) (log (- 1 probability) 2)))))))

;; in pseudocode its named importance
(defun attribute-information-gain (attribute examples
   previous-entropy)
  "the information gain of an attribute, which reflects its
     importance"
  (let ((possible-values (possible-attribute-values attribute
     examples))
        (total-entropy 0))
    ;; each value corresponds to a theoretical child of the
       node (if the attribute was to be split on)
    (loop for value in possible-values
          do (let* ((examples-with-that-value (
             examples-with-attribute-value examples attribute
             value))
                   (p-examples (positive-examples
                      examples-with-that-value))
                   (n-examples (negative-examples
                      examples-with-that-value)))
              (incf total-entropy (* (/ (length
                 examples-with-that-value) (length examples)) (
                 boolean-random-variable-entropy (/ (length
                 p-examples) (length examples-with-that-value))
                 )))))
    (- previous-entropy total-entropy)))

(defun positive-examples (examples)
  "get the examples whose goal is yes"
  (let ((positive-examples nil))
    (loop for example in examples
```

```
        do (if (equal (getf example 'goal) "yes")
               (push example positive-examples)))
   positive-examples))

(defun negative-examples (examples)
  "get the examples whose goal is no"
  (let ((negative-examples nil))
    (loop for example in examples
        do (if (equal (getf example 'goal) "no")
               (push example negative-examples)))
   negative-examples))

(defun examples-with-attribute-value (examples attribute value)
  "from a set of examples, get the ones with an attribute with
     a specific value"
  (let ((examples-with-that-attribute-and-value nil))
    (loop for example in examples
        do (if (equal (getf example attribute) value)
               (push example
                  examples-with-that-attribute-and-value)))
   examples-with-that-attribute-and-value))

(defun attributes-of-example (example)
  "list of attributes of example"
  (loop for (key value) on example by #'cddr
       collect key))
```

```
(defparameter
   learning-examples
  '((alt "yes" bar "no" fri "no" hun "yes" pat "some" price "
     $$$" rain "no" res "yes" type "french" est "0-10" goal "
     yes")
   (alt "yes" bar "no" fri "no" hun "yes" pat "full" price "$"
       rain "no" res "no" type "thai" est "30-60" goal "no" )
   (alt "no" bar "yes" fri "no" hun "no" pat "some" price "$"
      rain "no" res "no" type "burger" est "0-10" goal "yes")
   (alt "yes" bar "no" fri "yes" hun "yes" pat "full" price "$
      " rain "yes" res "no" type "thai" est "10-30" goal "yes"
      )
   (alt "yes" bar "no" fri "yes" hun "no" pat "full" price "
      $$$" rain "no" res "yes" type "french" est ">60" goal "
```

```
      no" )
   (alt "no" bar "yes" fri "no" hun "yes" pat "some" price "$$
      " rain "yes" res "yes" type "italian" est "0-10" goal "
      yes")
   (alt "no" bar "yes" fri "no" hun "no" pat "none" price "$"
      rain "yes" res "no" type "burger" est "0-10" goal "no" )
   (alt "no" bar "no" fri "no" hun "yes" pat "some" price "$$"
       rain "yes" res "yes" type "thai" est "0-10" goal "yes")
   (alt "no" bar "yes" fri "yes" hun "no" pat "full" price "$"
       rain "yes" res "no" type "burger" est ">60" goal "no" )
   (alt "yes" bar "yes" fri "yes" hun "yes" pat "full" price "
      $$$" rain "no" res "yes" type "italian" est "10-30" goal
       "no" )
   (alt "no" bar "no" fri "no" hun "no" pat "none" price "$"
      rain "no" res "no" type "thai"  est  "0-10" goal "no" )
   (alt "yes" bar "yes" fri "yes" hun "yes" pat "full" price "
      $" rain "no" res "no" type "burger" est "30-60" goal "
      yes")))

(decision-tree-learning learning-examples (remove 'goal (
   attributes-of-example (car learning-examples))) nil)
```